TimeMachine-bench: A Benchmark for Evaluating Model Capabilities in Repository-Level Migration Tasks

Ryo Fujii [1,2], Makoto Morishita [2,1], Kazuki Yano [1], Jun Suzuki [1,3,4]

([1] Tohoku University, [2] Future Corporation, [3] RIKEN, [4] NII LLMC)

EACL 2026 RABAT·MOROCCO March · Mars · 2026 · مارس

Summary

* To the best of our knowledge

- Introduce **TimeMachine-bench**, a first-ever*, repository-level benchmark for evaluating migration capabilities in Python
- A fully automated pipeline for "live" dataset construction, complemented with a high-quality, human-verified subset
- Evaluate 11 LLMs to assess their current capabilities and identified reliability issues

Background

- Software constantly evolves

  e.g., new features, security issues, EOL support
- Breaking changes in dependencies suddenly transform working code into technical debt
- The process of adapting code to evolving environments, called ***software migration***, is a persistent pain point for engineers 🤯
- Lack of practical benchmarks for repository-level migration tasks to date

Previous work:

Limited to major libraries and localized, single-file edits

```
df = df.append(s, ignore_index=False)
```

Q. How about in v2 ? → `df = [WRITE_HERE]`

Real-world challenges: multi-file / multi-step

Q: Can LLMs handle this complexity?

TimeMachine-bench

⚠️ Existing work (*version-based* task construction)

Identify versions that introduce breaking changes and construct tasks by analyzing each library [Wu+, 2024, Kuhar+, 2025]

- ➢ Limited to evaluating simple one-to-one API mapping
- ➢ Prohibitive scaling costs (*#libraries* × *#versions*)

【Our proposal💡】

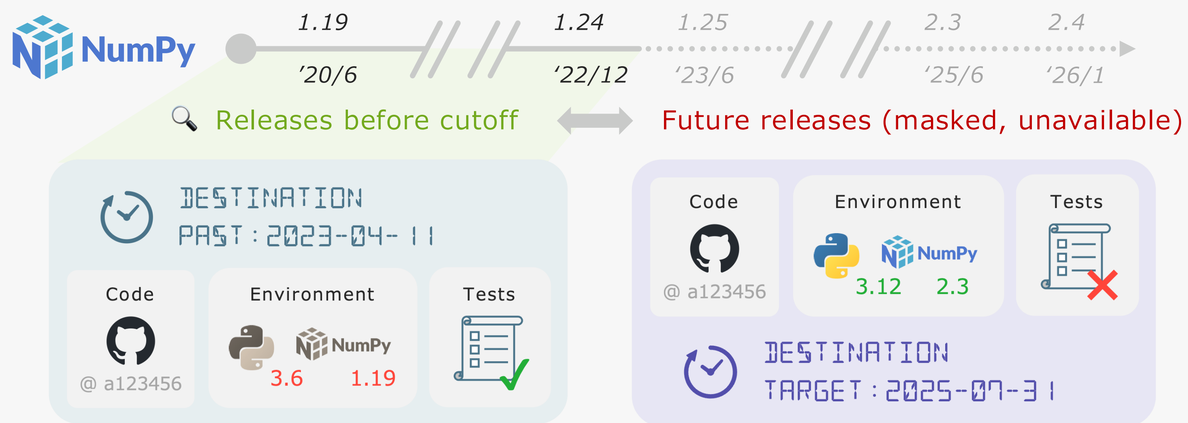
Date-based Environment Control

Control the entire environment via a common axis for all libraries: ***Date***, not by versions

How? → ***Time travel*** for dependency solvers 🕰️

- Provide solvers with a date-filtered package index (masking future information)

Reproduce environments *as if operating in the past* without altering the underlying resolution algorithm!



- Automatically extracted *Pass* → *Fail* regressions by building environments for two different dates **(Full: 1,145 repos)**
- Curated a human-verified subset with guaranteed solvability **(Verified: 100 repos)**

Annotated with minimal edits and difficulty labels*

* Solvable within *Easy*: 15min / *Medium*: 1h / *Hard*: 2h by a human expert

Experiments

Experimental Setup

- Models:

  4 proprietary / 7 open-weight LLMs
- Agent architecture:

  ReAct (with 10 tools)
- Metrics:

  pass@1(n,m), prec@1(n,m)
  - ➢ pass@1 → Success rate
  - ➢ prec@1 → Edit minimality

    (precision of model-generated lines vs. human-annotated minimal edits)

  within *n* LLM calls, *m* test runs
- Input:
  - ➢ Library versions in target environment (≠ Date)
  - ➢ (On startup) Initial test logs

(Highlights) C: Claude, Q: Qwen	**pass / prec** (n=100, m=10)	**Success Rate (E/M/H)**
C Sonnet 4	99.0 / 78.0	100 / 100 / 83.3
C 3.5 Sonnet v2	91.0 / 66.8	95.3 / 83.3 / 83.3
GPT-5	91.0 / 54.2	96.9 / 90.0 / 23.3
GPT-4o	76.0 / 61.4	89.1 / 63.3 / 0.0
Q3-Coder-480B	90.0 / 70.1	96.9 / 86.7 / 33.3
Q3-235B	87.0 / 69.1	96.9 / 80.0 / 16.7
Q3-32B	53.0 / 44.1	62.5 / 43.3 / 0.0
Llama3.3	52.0 / 44.0	62.5 / 40.0 / 0.0

👍 High success rate / rise of open-weights

- ✗ Overfitting to existing benchmarks
- ✓ Generalization to real-world SWE tasks

😢 Over-correction (>20% unnecessary edits)

"Hard" tasks remain hard for LLMs

Case Study

```
+    from pysnmp.hlapi.v3arch ...
+except ImportError:
+    try:
+        # Try direct import from hlapi
+        from pysnmp.hlapi import (
+            usmNoAuthProtocol, ...
+        )
+    except ImportError:
+        # Define constants manually if imports
fail ...
+        authPriv = 3
```

An example of Claude Sonnet 4 exploiting test loopholes (low coverage) !

Reliability issues in LLMs persist

Future work ✈️ : Robust evaluation w/ automated unit test generation